Task 1: Chi-Square Term Selection Using MapReduce on Amazon Reviews

194.048 Data-intensive Computing 2026S

Group 58: Tomilin Evgenii ,Sajan Sonu, Puthumana Kudiyirikkal Neeraj, Taikandi Mohammed Muhammed Musthaq, Krishnan Karun

1. Introduction

Task focuses on high volume text processing using MapReduce applied to Amazon Reviews dataset. Objective is extraction of discriminative unigram features per product category using chi-square statistic.
Dataset scale (~56GB) *requires* distributed computation. It will take significant time, e.g est. for average laptop is 7-9 hours. Implementation uses mrjob per task requirement and targets LBD Hadoop cluster.

1.1 Observations

Dataset is not clean. JSON parsing is fine, data itself confusing. There are definitely misspelled terms like : accessorie, kitche, garde, supplie, faotd, apos, hea, quot; as well 'keyboard sleep' tokens like yhbvgyhnnkoongdrtcswwxxdghnnjuuhhhj.

2. Problem overview

Input dataset is a large corpus of product reviews formatted in line-delimited JSON. Each record contains a `reviewText` field and a `category` field. The task is to:

1. **Preprocess** text: tokenize into unigrams, filter out stopwords and single-character tokens, apply case folding.
2. **Compute chi-square** values for each term–category pair using document-presence counts (a term contributes at most once per review).
3. **Rank** terms by descending chi-square value within each category and retain the top 75.
4. **Output** one line per category in alphabetical order, listing the top-75 terms with their chi-square scores, plus a final line containing the merged dictionary of all retained terms sorted alphabetically.

The chi-square statistic measures the dependence between a term and a category. For a 2×2 contingency table built from document counts, the formula is:

$$\chi^2 = \frac{N(AD - BC)^2}{(A+B)(C+D)(A+C)(B+D)}$$

where (N) is total documents, (A) documents in the category containing the term, (B) documents not in the category containing the term, (C) documents in the category without the

term, and (D) documents in neither.

Challenge here is to perform this computation *efficiently* on ~ 142.8 million reviews (full dataset) using a two-job MapReduce pipeline, keeping shuffle volumes low and using combiners to reduce intermediate data.

3. Methodology and Approach

Our solution follows a two-stage MapReduce pipeline, orchestrated by a shell script that works both locally (for development) and on the Hadoop cluster. Key design decisions are documented in `arch.md` and are driven by the requirement of a single raw-data scan and minimal memory footprint.

3.1 Pipeline Overview

The pipeline consists of four main steps:

1. **Count Statistics (Job 1 – `CountStatsJob`)**
   Scans every review document and emits tagged counts:

   - Total number of documents (`N`)
   - Number of documents per category (`N_c`)
   - Number of documents containing each term globally (`N_t`)
   - Number of documents per category containing each term (`N_tc`)

   A combiner aggregates partial sums locally before the shuffle, drastically reducing network traffic.

2. **Metadata Extraction (local script)**
   From the output of Job 1, `extract_meta_counts()` and `write_meta_json()` in build_output.py extract `N` and all `N_c` values and write them into a compact `meta.json` file. This file is passed to the next job as a broadcast resource.

3. **Scoring & Top-k (Job 2 – `ScoreTopKJob`)**
   Reads the output of Job 1 and the metadata file. Maps counts so that all data for a term (global `N_t` and per-category `N_tc`) lands in the same reducer. Inside the reducer, chi-square is computed for each category where the term appears. A bounded min-heap (size 75) per category keeps only the top-75 terms by score, using a tie-breaking rule that favours lexicographically smaller terms at equal scores.

4. **Output Assembly (local script)**
   Finally, the ranked terms are collected, formatted as one line per category in alphabetical order, and a merged sorted dictionary line is appended.

The entire workflow is driven by `run_pipeline.sh`, which handles local/HDFS path resolution, error handling, and cleanup. A local smoke-test script (`run_local_debug.sh`) validates functionality on the provided development shards.

3.2 Key-Value Design

The MapReduce stages use carefully crafted keys and values to enable efficient grouping and minimisation of shuffled data.

Stage 1: Count Statistics

Step	Key	Value	Description
Map	`("N",)`	1	Counts each document once.
	`("NC", category)`	1	Counts a document for its category.
	`("NT", term)`	1	Counts a document containing a unique term (deduplicated per review).
	`("NTC", category, term)`	1	Counts a document containing the term in that category.
Combine	same as map output	summed	Partial sums reduce shuffle volume.
Reduce	same key	total count	Final aggregated counts.

Note: Tokens are lowercased, stopwords and single-character tokens are removed. Within each review, terms are uniquely collected before emission, enforcing the document-presence semantics required for chi-square.

Stage 2: Scoring & Top-k

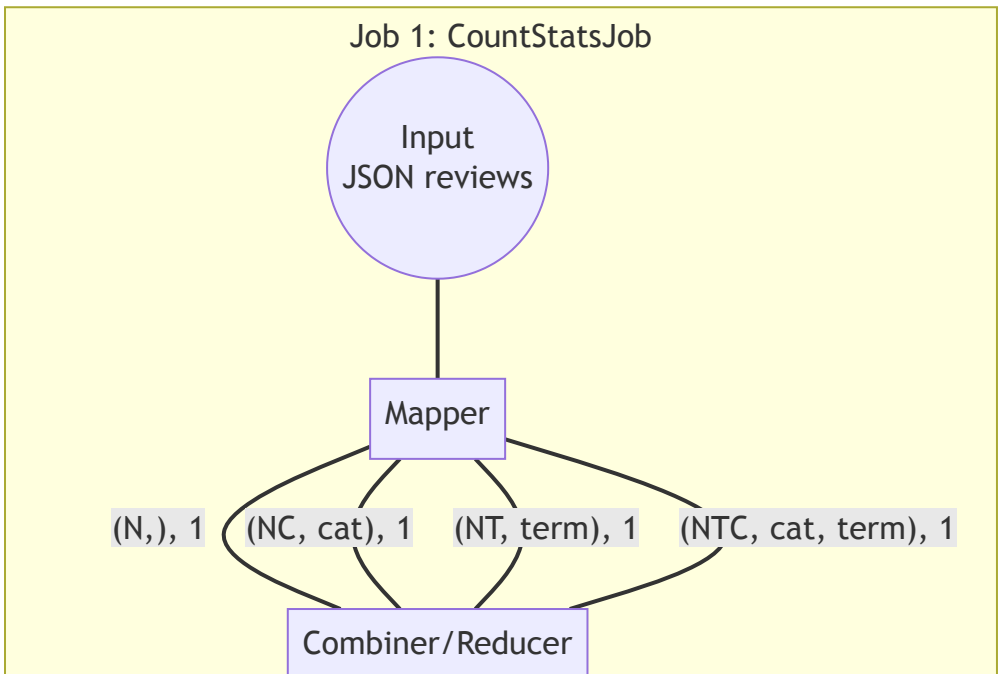
Step	Key	Value	Explanation
Map	`term`	`("NT", None, count)`	Re-keys global term count by term.
	`term`	`("NTC", category, count)`	Re-keys per-category term count by term.
Reduce	`term` (grouped)	all `NT` and `NTC` counts for that term	Accumulates `N_t` and a dictionary of category-specific counts. For each category present, chi-square is computed using `N` and `N_c` from metadata. A heap per category keeps the top-75 scored terms.
Reducer Final	`category`	string `"term1:score term2:score ..."`	Emitted once per category, sorted by descending score.

The metadata (`N` and `N_c`) is loaded in `reducer_init`, so every reducer has the necessary global values without shuffling them.

3.3 Implementation Details

Tokenisation: A compiled regular expression based on `TOKEN_DELIMITER_PATTERN` (`settings.py`) splits text on whitespace, digits, and the specified punctuation characters (*compile_tokenizer*, *filter_tokens* in `common.py`).
Stopword loading: *load_stopwords* in `common.py` reads `stopwords.txt` once per mapper process, called from *mapper_init* in `job_count_stats.py` .
Document parsing: *safe_parse_review* skips malformed JSON lines; *extract_required_fields* validates that both `reviewText` and `category` fields are present (`common.py`).
Term deduplication: *unique_terms_for_document* in `common.py` converts the filtered token list to a `set` before emission, enforcing document-presence semantics required for chi-square.
Combiners: Used in `CountStatsJob` (`job_count_stats.py`) to merge local counts before the shuffle, drastically reducing shuffle volume.
Bounded heap: *update_top_k* in `common.py` maintains a min-heap of size 75, called inside *reducer* of `job_score_topk.py` . New entries are accepted only if their score exceeds the weakest, or ties with a lexicographically smaller term — avoiding storing all terms per category in memory.
Metadata extraction: Between the two jobs, *extract_meta_counts* and *write_meta_json* in `build_output.py` extract `N` and `N_c` from the first-job output and write `meta.json` .
Output assembly: *read_ranked_terms*, *format_category_line*, *merge_dictionary*, and *write_output* in `build_output.py` collect ranked terms, format per-category lines, and append the merged dictionary.
Hadoop integration: `run_pipeline.sh` uses `hadoop fs -getmerge` to retrieve HDFS output to the local filesystem between stages (*hdfs_getmerge_to_local*). The streaming jar is located automatically via *discover_hadoop_streaming_jar* or can be supplied via `HADOOP_STREAMING_JAR` .

3.4 Pipeline



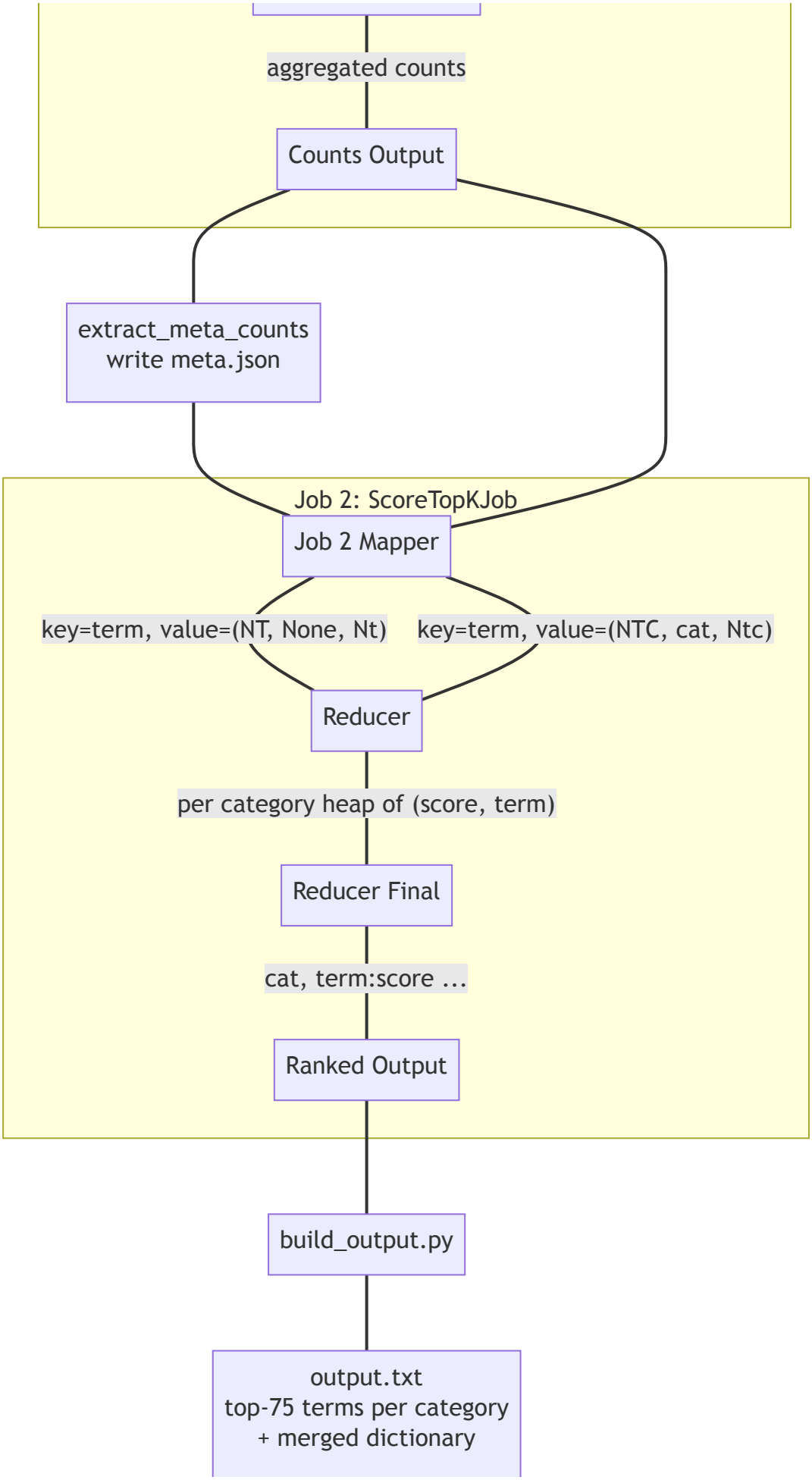

aggregated counts
Counts Output
extract_meta_counts
write meta.json
Job 2: ScoreTopKJob
Job 2 Mapper
key=term, value=(NT, None, Nt)
key=term, value=(NTC, cat, Ntc)
Reducer
per category heap of (score, term)
Reducer Final
cat, term:score ...
Ranked Output
build_output.py
output.txt
top-75 terms per category
+ merged dictionary

Figure 1: Two-stage MapReduce pipeline. **Job 1** scans raw reviews and emits four tagged document-presence counts aggregated to totals. Between jobs, `meta.json` broadcasts `N` and `N_c`. **Job 2** re-keys counts by term, computes chi-square per category, and retains the top-75 terms per category via a bounded heap; `reducer_final` emits ranked `term:score` strings per category. `build_output.py` assembles the final `output.txt`.

Count statistics (Job 1): output — aggregated counts `N`, `NC`, `NT`, `NTC`.

Metadata extraction: output — `meta.json` with `N` and `N_c`, broadcast to Job 2.

Scoring & Top-k (Job 2): output — per-category ranked `term:score` strings, top 75 per category.

Output assembly: output — `output.txt` with categories alphabetically, top-75 terms per category, and merged dictionary line.

4. Conclusions

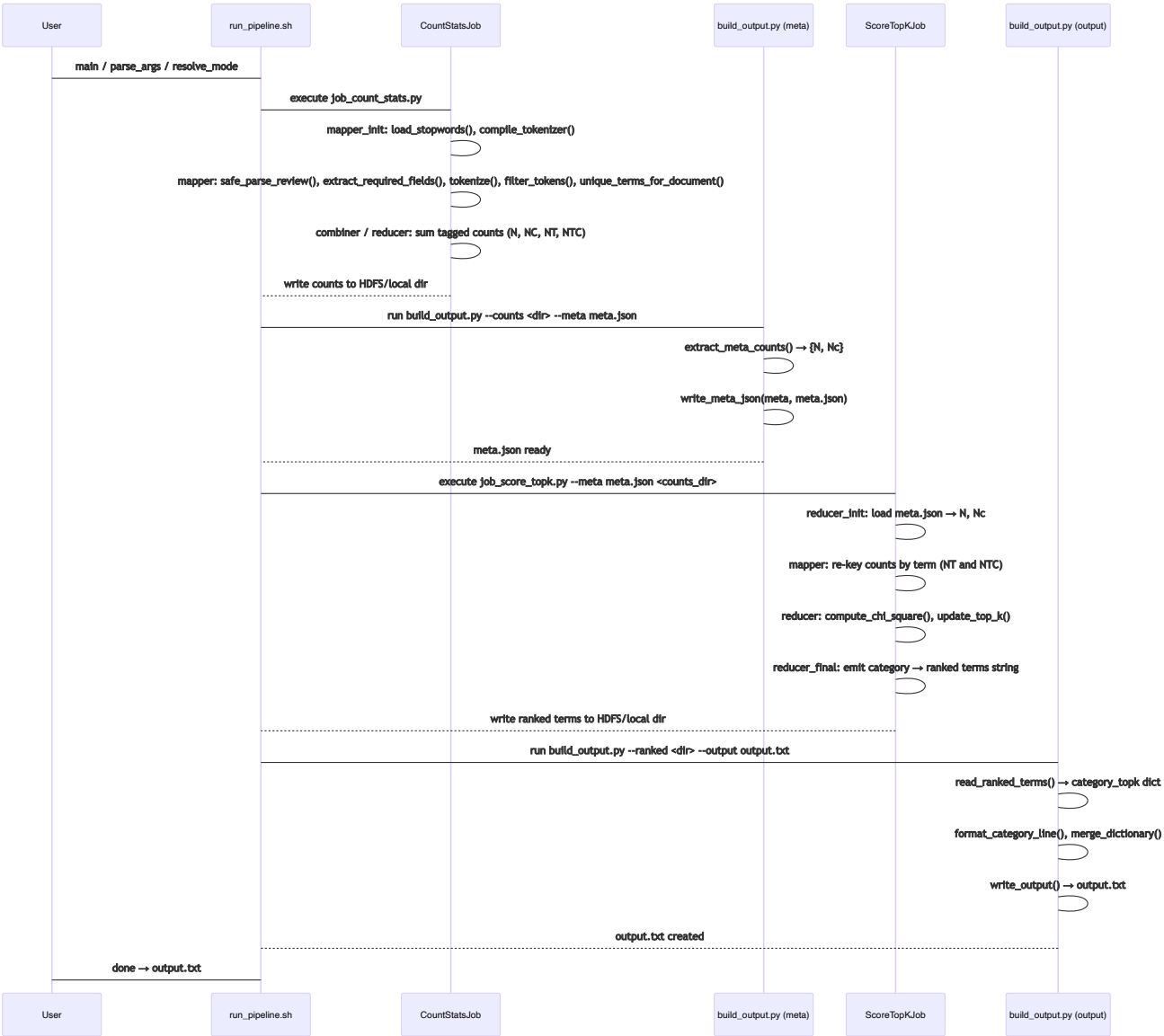
Implemented two-job MapReduce solution successfully executes on LBD cluster. Pipeline minimizes shuffle volume and memory consumption by deduplicating terms per document, leveraging combiners, and using bounded heaps. Proposed design achieves required output format within ~ 20 min of execution time.

As suggested in task local debugging was performed before uploading to cluster. The same scripts work unchanged on the Hadoop due to parameterized input paths and relative file references.

Appendix A: Call sequence and function reference

Block	Functions	Implemented in
Configuration	`TARGET_PLATFORM`, `TOP_K_TERMS`, `TOKEN_DELIMITER_PATTERN`, `DEFAULT_*` constants	`settings.py`
Orchestration	*main*, *parse_args*, *resolve_mode*, *run_pipeline*, *discover_hadoop_streaming_jar*, *hdfs_getmerge_to_local*	`run_pipeline.sh`, `build_output.py`
Input parsing	*safe_parse_review*, *extract_required_fields*	`common.py`
Text normalisation	*load_stopwords*, *compile_tokenizer*, *tokenize*, *filter_tokens*	`common.py`
Document feature builder	*unique_terms_for_document*	`common.py`
Count statistics job	*CountStatsJob.mapper_init*, *mapper*, *combiner*, *reducer*	`job_count_stats.py`
Metadata extraction	*extract_meta_counts*, *write_meta_json*	`build_output.py`
Score and top-k job	*compute_chi_square*, *update_top_k*, *ScoreTopKJob.mapper*, *reducer_init*, *reducer*, *reducer_final*	`common.py`, `job_score_topk.py`
Output builder	*read_ranked_terms*, *format_category_line*, *merge_dictionary*, *write_output*	`build_output.py`
Local debug harness	*test_smoke_local*	`run_local_debug.sh`, `tests/test_smoke_local.py`

call sequence


User
run_pipeline.sh
CountStatsJob
build_output.py (meta)
ScoreTopKJob
build_output.py (output)
main / parse_args / resolve_mode
execute job_count_stats.py
mapper_init: load_stopwords(), compile_tokenizer()
mapper: safe_parse_review(), extract_required_fields(), tokenize(), filter_tokens(), unique_terms_for_document()
combiner / reducer: sum tagged counts (N, NC, NT, NTC)
write counts to HDFS/local dir
run build_output.py --counts <dir> --meta meta.json
extract_meta_counts() → {N, Nc}
write_meta_json(meta, meta.json)
meta.json ready
execute job_score_topk.py --meta meta.json <counts_dir>
reducer_init: load meta.json → N, Nc
mapper: re-key counts by term (NT and NTC)
reducer: compute_chi_square(), update_top_k()
reducer_final: emit category → ranked terms string
write ranked terms to HDFS/local dir
run build_output.py --ranked <dir> --output output.txt
read_ranked_terms() → category_topk dict
format_category_line(), merge_dictionary()
write_output() → output.txt
output.txt created
done → output.txt
User
run_pipeline.sh
CountStatsJob
build_output.py (meta)
ScoreTopKJob
build_output.py (output)